



An effective hybrid honey bee mating optimization algorithm for balancing mixed-model two-sided assembly lines



Biao Yuan^a, Chaoyong Zhang^{b,*}, Xinyu Shao^b, Zhibin Jiang^a

^a Department of Industrial Engineering and Management, School of Mechanical Engineering, Shanghai Jiao Tong University, Shanghai 200240, China

^b State Key Laboratory of Digital Manufacturing Equipment and Technology, School of Mechanical Science and Engineering, Huazhong University of Science and Technology, Wuhan, Hubei 430074, China

ARTICLE INFO

Available online 2 August 2014

Keywords:

Mixed-model

Two-sided assembly line

Honey bee mating optimization

Simulated annealing

ABSTRACT

Mixed-model two-sided assembly lines are widely used in a range of industries for their abilities of increasing the flexibility to meet a high variety of customer demands. Balancing assembly lines is a vital design issue for industries. However, the mixed-model two-sided assembly line balancing (MTALB) problem is NP-hard and difficult to solve in a reasonable computational time. So it is necessary for researchers to find some efficient approaches to address this problem. Honey bee mating optimization (HBMO) algorithm is a population-based algorithm inspired by the mating process in the real colony and has been applied to solve many combinatorial optimization problems successfully. In this paper, a hybrid HBMO algorithm is presented to solve the MTALB problem with the objective of minimizing the number of mated-stations and total number of stations for a given cycle time. Compared with the conventional HBMO algorithm, the proposed algorithm employs the simulated annealing (SA) algorithm with three different neighborhood structures as workers to improve broods, which could achieve a good balance between intensification and diversification during the search. In addition, a new encoding and decoding scheme, including the adjustment of the final mated-station, is devised to fit the MTALB problem. The proposed algorithm is tested on several sets of instances and compared with Mixed Integer Programming (MIP) and SA. The superior results of these instances validate the effectiveness of the proposed algorithm.

© 2014 Elsevier Ltd. All rights reserved.

1. Introduction

The assembly line is a traditional and important means of mass production or customization production. Since it is introduced by Henry Ford into his automobile plants, the assembly line has been developed a lot. According to the features of products and technical or operational requirements, assembly lines can be categorized into one-sided assembly lines (one-ALs) and two-sided assembly lines (two-ALs) [1]. One-ALs can use only one side of the lines to handle small-sized products, whereas two-ALs can use both (left and right) sides in parallel to produce large-sized products. Compared with one-ALs, two-ALs have several advantages as follows [2,3]: (i) shorten the line length; (ii) reduce the amount of throughput time; (iii) reduce material handling, worker movement, and setup time; and (iv) reduce the cost of tools and fixtures.

* Correspondence to: Room B603, Advanced Manufacturing Building, Huazhong University of Science and Technology, Wuhan, Hubei 430074, China.
Tel.: +86 27 87556063; fax: +86 27 87557765 0.

E-mail address: zcylust@mail.hust.edu.cn (C. Zhang).

Balancing assembly lines is a difficult problem for most of the industries. It refers to assigning a finite set of tasks to a set of stations to reach the desired level of performance or the optimization objectives. Each task has an operation time and a set of precedence relations. Due to the use of both sides of the lines, the tasks of two-ALs have additional operation direction restrictions. The directions can be classified into three types: left side (L), right side (R) and either side (E). A two-sided assembly line is illustrated in Fig. 1. Two directly facing stations, e.g., Stations 1 and 2, are called a mated-station, and one of them calls the other a companion. Some unavoidable idle times (shaded rectangles) can be seen in Fig. 1. This phenomenon is called *sequence-dependent finish time of tasks* [3] or *interference* [4]. It always happens in the same mated-station. Considering Tasks 3 and 4 in Mated-station 1, if Task 3 should be finished before Task 4, the idle time between Tasks 1 and 4 is unavoidable.

Single-model assembly lines are designed to produce standardized homogeneous products. However, they are unsuitable for today's competitive market because of the high variety of customer requirements. In order to improve the flexibility to adapt to this situation, mixed-model assembly lines are widely applied in a range of industries, including the automobile industry, consumer

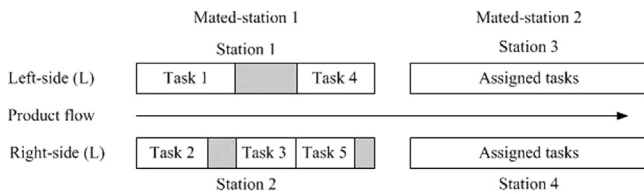


Fig. 1. Configuration of a two-sided assembly line.

electronics, white goods, furniture and clothing [31]. Mixed-model assembly lines refer to producing two or more products with similar production characteristics in the line. Taking the automobile industry which uses mixed-model two-sided assembly lines as an example, a wide choice of components (e.g., manual or electric sunroof, air conditioning yes/no, color of the car body) about a car is selectable by customers, so manufacturers need to deal with a (theoretical) product variety which exceeds several billions of models [31]. To maintain such a highly diversified product portfolio without harming the benefits of an efficient flow-production, mixed-model lines are employed. And balancing mixed-model lines is very important to take advantage of them, because unbalanced lines may incur unnecessary cost (e.g., extra stations, operators, and tools) and workers' dissatisfaction with different workloads. Although some approaches have been proposed to balance single-model two-ALs, they cannot be used directly for mixed-model ones in which different production models should be considered. And the mixed-model two-sided assembly line balancing problem is more complex than the single-model one-sided one which proves to be NP-hard [5], since different models and the sequence-dependent finish time of tasks should be considered in the former one. So it is necessary for researchers to find some efficient approaches to solve this problem.

This paper focuses on a new meta-heuristic algorithm named honey bee mating optimization (HBMO) to address the mixed-model two-sided assembly line balancing (MTALB) problem with the objective of minimizing the number of mated-stations and total number of stations. HBMO is a population-based algorithm inspired by the mating process in the real colony [6]. It has been applied to many optimization problems successfully. And this paper is the first attempt to adopt the algorithm to solve the MTALB problem.

The remainder of this paper is organized as follows. Section 2 presents the literature review of two-sided assembly line balancing problems. Section 3 describes the MTALB problem in detail. Section 4 introduces the principle of the HBMO algorithm. Section 5 gives the hybrid HBMO algorithm for the problem. Section 6 discusses the performance of the proposed algorithm. The final Section 7 concludes the paper.

2. Literature review

Since Salveson proposed the assembly line balancing problem in 1955, many studies on the problem have been reported in the literature. The recent reviews of these studies have been given by Scholl and Becker [5], Boysen et al. [7], and Tasan and Tunali [8]. Although many algorithms for balancing assembly lines are available, they mostly concentrate on one-ALs. Little attention has been paid to two-ALs.

Bartholdi [2] first proposed the two-ALB problem, and developed an interactive computer program based on a modified version of the 'first fit' heuristic. Kim et al. [9] used a genetic algorithm (GA) to solve the two-ALB problem with positional constraints. Lee et al. [3] designed a group assignment procedure,

which assigned a group of tasks rather than a single task, to balance two-ALs to maximize the indexes of the work relatedness (IWR) and the work slackness (IWS). Lapierre and Ruiz [10] proposed an enhanced priority-based heuristic to solve the specific two-ALB problem from an industrial case. Baykasoglu and Dereli [11] utilized an ant colony optimization (ACO) algorithm to solve the two-ALB problem with zoning constraints. Wu et al. [12] and Hu et al. [13] proposed branch-and-bound (B & B) algorithms to solve the problem, respectively. Hu et al. [14] developed a station-oriented enumerative algorithm, which was integrated with the Hoffmann heuristic, to solve the two-ALB problem. Ozcan and Toklu [15] proposed a tabu search (TS) algorithm for the two-ALB problem to maximize the line efficiency and minimize the smooth index. Ozcan and Toklu [16] proposed a pre-emptive mixed integer goal programming (MIGP) model for precise goals and a fuzzy mixed integer goal programming (FMGIP) model for imprecise goals to solve two-ALB problems with multiple objectives. Kim et al. [33] proposed the genetic algorithm (GA) to solve the type-II two-ALB problem, which was to minimize the cycle time for a given number of stations. Ozcan [34] considered the two-ALB problem with stochastic task times (STALB) in his paper, constructed a chance-constrained, piecewise-linear, mixed integer program (CPMIP), and solved the problem by SA. Ozcan and Toklu [17] developed a mixed integer programming (MIP) model and proposed a heuristic approach based on the COMSOAL method for the two-ALB problem with sequence-dependent setup times. Ozcan et al. [18] proposed a new type of assembly lines called the parallel two-sided assembly line, and utilized the tabu search (TS) algorithm to balance it. Ozbakir and Tapkan [19] applied the bee algorithm (BA) for balancing two-ALs with zoning constraints, and obtained the majority of optimal or near optimal solutions of two-ALB instances with and without zoning constraints.

All the above studies are about single-model assembly lines. Only several studies have been reported on the MTALB problem in the literature. Simaria and Vilarinho [4] constructed a mathematical programming model for the MTALB problem. However, due to its high complexity, the model seemed impossible to be solved optimally. The ACO algorithm, named 2-ANTBAL, was developed to solve this problem. Ozcan and Toklu [20] presented a new mathematical model for the problem and solved it for small-sized problems. But large-sized problems were solved by the simulated annealing (SA) algorithm. Rabbani et al. [21] formulated a mixed integer programming (MIP) model for mixed-model two-sided assembly lines with multiple U-shaped layouts and designed a heuristic based on GA to solve this problem. Chutima and Chimklai [1] proposed the particle swarm optimization (PSO) algorithm with negative knowledge to solve the multi-objective mixed-model two-sided assembly line balancing problem. To our best knowledge, only the above four papers are about the MTALB problem.

3. Mixed-model two-sided assembly line balancing problems

A mixed-model two-sided assembly line is designed to produce a set of similar product models in any model order and mix by the operators who perform tasks on a set of mated-stations, which have a pair of stations directly opposite each other (left and right side) [4]. Each model has its own precedence relationship that can be described by a precedence graph. All the precedence graphs of the models can be merged into a single precedence diagram, termed the combined precedence graph [20]. Fig. 2 shows precedence graphs, task times and directions of two models (a), (b) and their combined precedence graph (c). Each cycle represents a task, and the arrow connecting two different tasks denotes their precedence relationship. Each task in the combined precedence

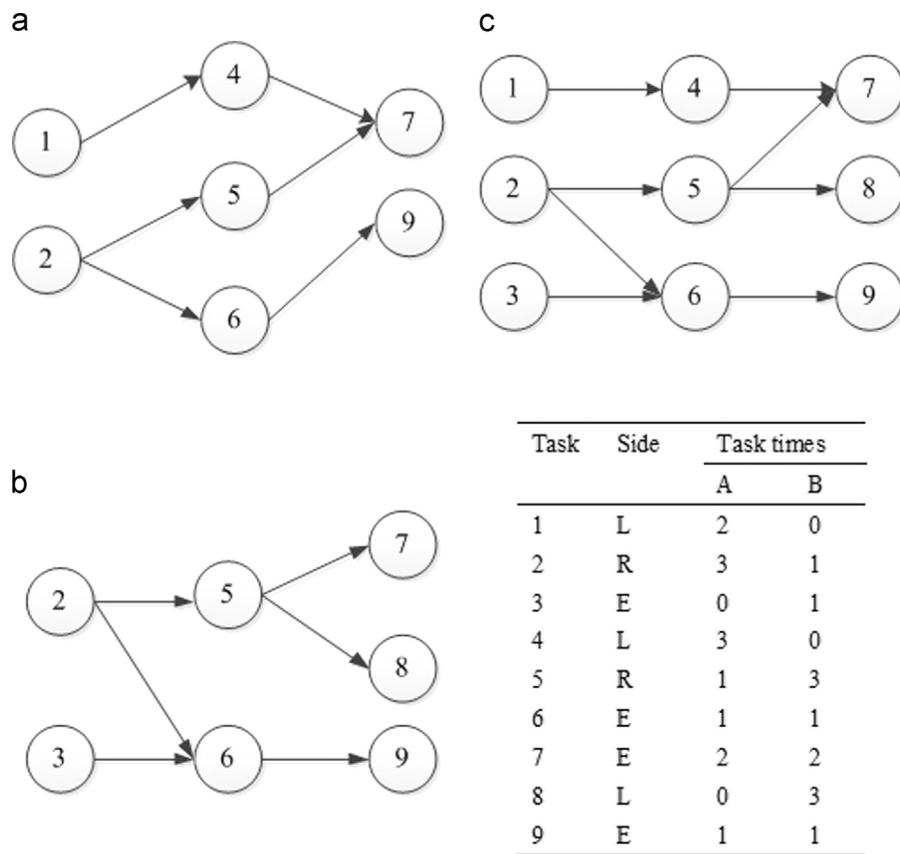


Fig. 2. Precedence graphs, task times and directions of two models.

graph has a different task time but the identical operation direction for different models. A task time of zero indicates that the corresponding model does not contain this task. The forecasted demand, over the given planning horizon P , for model m ($m \in M$) is D_m . The cycle time (CT) of the MTALB problem is calculated by Eq. (1), and the overall proportion of the number of units of model m being produced (q_m) is computed by Eq. (2) [4].

$$CT = P / \sum_{m \in M} D_m \quad (1)$$

$$q_m = D_m / \sum_{m \in M} D_m \quad \forall m \in M \quad (2)$$

The MTALB problem in this study includes the following assumptions [20]:

- (1) The product models with similar characteristics are produced in the line.
- (2) The operators perform their tasks in parallel on both sides of the line within a given cycle time.
- (3) The travel times of the operators are ignored.
- (4) No work-in-process (WIP) inventory is allowed.
- (5) Task times are deterministic and independent of stations.
- (6) The precedence graphs of different models are known previously.
- (7) The zoning constraints, positional constraints, and synchronous task constraints are not considered in the problem.

4. The principle of HBMO

In a typical colony, there are three types of bees including queens, drones and workers. Queens play a vital role in the colony

because they keep the colony working by producing broods, and one colony often has one queen. Drones are fathers of the colony and have just one duty which is to provide the queen with their sperms. Workers are responsible for the brood care and sometimes lay eggs.

During the mating process, the queen flies away from the nest, begins the mating flight, and then mates with the selected drones which follow it in the air. In each mating flight, the sperms supplied by the selected drones are stored in the queen's spermatheca to form the genetic pool. Each time a queen generates a new brood, it randomly chooses a sperm accumulated in its spermatheca. When new broods are produced, workers foster them.

Inspired by the above mating process, Abbass first proposes the honey bee mating optimization algorithm [6]. Similar to other population-based algorithms, HBMO also maintains a population of solutions to the problem throughout the optimization process. In the algorithm, the queen represents the current best solution; drones provide different genes to keep solutions diverse; and workers denote some local search methods which act to improve solutions. The mating flights can be regarded as a set of transitions in the state-space. The queen flies between different states in some speed, and mates with the drones encountered at each mating flight according to the probabilistic rule.

At the beginning of the flight, the queen is initialized with some energy content (i.e., speed and energy), and returns to its nest when its energy is less than a threshold value or its spermatheca is full. A drone mates with the queen probabilistically using the annealing function as follows [6]:

$$Prob(Q, D) = \exp\left(-\frac{\Delta(f)}{S(t)}\right) \quad (3)$$

where $Prob(Q, D)$ is the probability of adding the sperm of drone D to the spermatheca of queen Q ; $\Delta(f)$ is the absolute difference

between the fitness of D and the fitness of Q ; and $S(t)$ is the speed of the queen at time t . After each transition, the queen's speed $S(t)$ and energy $E(t)$ damp using the following equations [23]:

$$S(t+1) = \alpha S(t) \quad (4)$$

$$E(t+1) = \alpha E(t) \quad (5)$$

where α is a factor $\in (0, 1)$, the reduction coefficient of energy and speed after each transition.

A pseudo code of the HBMO algorithm [25] is presented as follows, while in the next Section the proposed algorithm for the MTALB problem is explained in detail.

```

Generate the initial population
Evaluate the whole population
Select the best bee as the queen
do  $i=1$ , number of mating flights
    Initialize the queen's spermatheca, energy and speed
    do while energy > threshold value and spermatheca is not full
        Select a drone
        if the drone meets the probabilistic condition
            then Put the sperm of the selected drone into the spermatheca
            endif
            speed and energy damp according to Eqs. (4) and (5)
            enddo
            do  $j=1$ , size of spermatheca
                Select a sperm from the spermatheca randomly
                Generate a brood using a crossover operator between the sperm and the queen
                Improve the new brood by a randomly selected worker
                if the brood's fitness is better than the queen's then
                    Replace the queen with the brood
                else if the brood's fitness is better than the worst fitness of the drones then
                    Replace the worst drone with the brood
                endif
            endif
        enddo
    enddo
Output the queen (the best solution)

```

HBMO has been employed to address several problems, including water resources optimization [22], probabilistic traveling salesman problem [23], financial classification problem [24], Euclidean traveling salesman problem [25], multi-objective placement of renewable energy resource problem [26], educational timetabling problem [27], and alternative design redundancy allocation problem [28]. It has demonstrated better performance than other algorithms in solving these problems. In this study, it is firstly applied to the MTALB problem.

5. The hybrid HBMO algorithm for the MTALB problem

The procedure of the hybrid HBMO (HHBMO) algorithm for the MTALB problem is described in the following subsections.

5.1. Encoding and decoding

The usage of decimal numbers is for the coding of genotypes or solutions in this study. The length of genotypes is equal to

the number of tasks. The numeral represents the task's number. The location of the numeral represents the precedence relationship between tasks. A genotype of the problem in Fig. 2 is illustrated in Fig. 3. It indicates the sequence of assigning tasks to stations, which is $\{1, 4, 2, 3, 5, 6, 9, 7, 8\}$.

The genotype only provides the information on the assignment sequence of tasks when tasks are assigned to stations, but not give the detailed configuration of the assembly line. So an improved decoding method based on the method of Ozcan and Toklu [17] is proposed to deal with this situation. The following notations are used in the decoding process.

i	the i th gene in a genotype
$h(i)$	the task corresponding to the i th gene of a genotype
I	set of tasks, $h(i) \in I$
k	the index of mated-stations
K	set of mated-stations, $k \in K$
m	the index of product models
M	set of product models, $m \in M$
A_L	set of tasks which should be performed at a left side station
A_R	set of tasks which should be performed at a right side station
A_E	set of tasks which should be performed at either side of a mated-station
$P(h)$	set of the immediate predecessors of task h
t_h^m	operation time of task h for model m
tf_h^m	finish time of task h for model m
wl_k^m	station load including idle time of the left side station of mated-station k for model m
wr_k^m	station load including idle time of the right side station of mated-station k for model m
CT	cycle time
N	length of a genotype
nm, nl, nr	number of mated-stations, left side stations and right side stations
ns	total number of left stations and right stations
SL_k	set of tasks in the left side station of mated-station k
SR_k	set of tasks in the right side station of mated-station k

The process for decoding a genotype is given as follows.

Step 1: Set $i=1$, $k=0$, $nm=0$, $nl=0$, $nr=0$, $wl_k^m = 0$ and $wr_k^m = 0$ for $\forall m \in M$.

Step 2: If i is greater than N , then go to Step 5.

Step 3: Assign task $h(i)$ for which

3.1 If task $h \in A_L$ then,

If $t_h^m + wl_k^m \leq CT$ and $t_h^m + tf_r^m \leq CT$ ($tf_r^m = \max\{tf_p^m | p \in P(h) \text{ have already been assigned to the right side station of the current mated-station } k\}$) for $\forall m \in M$, then assign task h to the left side station, set $tf_h^m = \max\{t_h^m + wl_k^m, t_h^m + tf_r^m\}$ and $wl_k^m = tf_h^m$ for $\forall m \in M$, $i=i+1$ and go to Step 2; otherwise, go to Step 4.

3.2 If task $h \in A_R$ then,

If $t_h^m + wr_k^m \leq CT$ and $t_h^m + tf_l^m \leq CT$ ($tf_l^m = \max\{tf_p^m | p \in P(h) \text{ have already been assigned to the left side station of the current mated-station } k\}$) for $\forall m \in M$, then assign task h to the right side station, set $tf_h^m = \max\{t_h^m + wr_k^m, t_h^m + tf_l^m\}$ and $wr_k^m = tf_h^m$ for $\forall m \in M$, $i=i+1$ and go to Step 2; otherwise, go to Step 4.

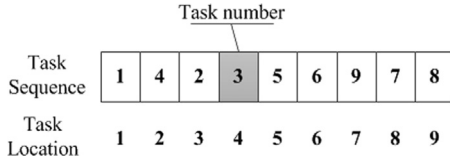


Fig. 3. A genotype.

3.3 If task $h \in A_E$ then,

If $\max\{wl_k^m, tf_r^m\} > \max\{wr_k^m, tf_l^m\}$ for a given $m \in M$, then go to Step 3.2; otherwise, go to Step 3.1.

Step 4: If the workload on either side of mated-station k is greater than CT , then open a new mated-station. If $wl_k^m > 0$ for $\exists m \in M$, then $nl = nl + 1$. If $wr_k^m > 0$ for $\exists m \in M$, then $nr = nr + 1$. Set $nm = nm + 1$, $k = k + 1$, $wl_k^m = 0$ and $wr_k^m = 0$ for $\forall m \in M$, and go to Step 2.

Step 5: If $wl_k^m > 0$ for $\exists m \in M$, then $nl = nl + 1$. If $wr_k^m > 0$ for $\exists m \in M$, then $nr = nr + 1$. And set $nm = nm + 1$.

Step 6: Adjust the assignment of the tasks in the final mated-station for which

6.1 If $\sum_h t_h^m \leq CT$ for all $h \in (SL_{nm} \cup SR_{nm})$ then,

6.1.1 If $h \in A_L$ for $\exists h \in SL_{nm}$ and $h \in A_E$ for $\forall h \in SR_{nm}$, assign all tasks $h \in SR_{nm}$ to the left side station, and set $nr = nr - 1$.

6.1.2 If $h \in A_R$ for $\exists h \in SR_{nm}$ and $h \in A_E$ for $\forall h \in SL_{nm}$, assign all tasks $h \in SL_{nm}$ to the right side station, and set $nl = nl - 1$.

6.1.3 If all the tasks $h \in A_E$, assign all tasks $h \in SR_{nm}$ to the left side station, and set $nr = nr - 1$.

Step 7 Output nm , ns , nl , and nr , then stop.

The differences between the proposed method and the one used by Ozcan and Toklu [17] are: (i) for task $h \in A_E$, both methods assign h to the side which can start the task earlier, but when the starting times of both sides are equal, the proposed method assigns it to the left side by default (Step 3.3), whereas the latter one randomly assigns it to the left or right side; and (ii) in the last mated-station, the tasks in the left side and right side station may be merged into one station (Step 6), and this situation is considered in this method but not in the latter one or other similar ones (Ozbakir and Tapkan [19], Ozcan and Toklu [20]).

The search space is largely reduced by the proposed method because of the rule that assigning the E-type tasks to the left side when the starting times are equal. To enlarge the search space, the above decoding process runs once again under the condition that if the starting times are equal then the E-type tasks are assigned to the right side (see in Step 3.3, change $\max\{wl_k^m, tf_r^m\} > \max\{wr_k^m, tf_l^m\}$ to $\max\{wl_k^m, tf_r^m\} \geq \max\{wr_k^m, tf_l^m\}$). The final decoded result for a genotype is the one that can achieve the better fitness.

5.2. Fitness function

The objective of the MTALB problem is to minimize the number of mated-stations and total number of stations for a given cycle time. However, it cannot distinguish between different solutions sometimes. So the weighted line efficiency (WLE) and weighted smoothness index (WSI) are considered in the fitness function [20]. And the linear weighted sum method is adopted to combine the four objectives into a single one. The calculation of the fitness value for a decoded genotype is given as follows:

$$f = w_{nm}nm + w_{ns}(nl + nr) + w_{WLE} \frac{WLE_0}{WLE} + w_{WSI} \frac{WSI}{WSI_0} \quad (6)$$

$$WLE = \left(\frac{\sum_{m \in M} q_m \left(\sum_{h \in I} t_h^m \right)}{CT(nl + nr)} \right) 100 \quad (7)$$

$$WSI = \sqrt{\frac{\sum_{m \in M} q_m \left(\sum_{k \in K} ((wl_k^m - wlr_{max})^2 + (wr_k^m - wlr_{max})^2) \right)}{(nl + nr)}} \quad (8)$$

where w_{nm} is the weighted coefficient of the number of mated-stations (nm), w_{ns} is the weighted coefficient of the total number of left side stations (nl) and right side stations (nr), w_{WLE} is the weighted coefficient of the weighted line efficiency, w_{WSI} is the weighted coefficient of the weighted smoothness index, WLE_0 and WSI_0 are obtained from the initial solution, wlr_{max} is the maximum workload among stations. Other notations can be referred to Sections 3 and 5.1. In our study, w_{nm} , w_{ns} , w_{WLE} and w_{WSI} are set equal to 10, 5, 1 and 1, respectively.

5.3. Generate the population

A feasible solution or genotype means that the sequence of tasks in the solution satisfies precedence constraints. The generation of a feasible solution mainly contains two steps: firstly, randomly generate a task sequence regardless of the precedence relationships; secondly, transform this sequence into a feasible one by the binary tree method. The binary tree method includes constructing a binary tree on the basis of the precedence relationships, listing the feasible solution according to the inorder traversal rank of the constructed binary tree. The details of the binary tree method can be referred to the paper of Tseng [29]. More feasible solutions can be generated by the above method, and constitute the initial population.

5.4. Crossover operator

In this study, a modified one-point crossover operator [29,30] is utilized for the queen and the selected drones (sperms), and the precedence constraints are also maintained during the duplication and exchange process. So this process can keep solutions still feasible. There are five steps during the process as follows.

Step 1: Select one drone's sperm from the spermatheca.

Step 2: Randomly generate an integer l , which implies the location of the crossover point in the genotype.

Step 3: The crossover point divides the genotype into two parts, *front* and *rear*. Duplicate the tasks located in the *front* part of the genotypes of the queen and the selected drone, and place them in the corresponding locations of brood1 and brood2.

Step 4: According to the precedence relationships between tasks in the drone's genotype, duplicate the tasks located in the *rear* part of the genotype of the queen, and place them in the corresponding locations of brood1. In the same way, duplicate the tasks located in the *rear* part of the genotype of the drone, and place them in the corresponding locations of brood2 according to the precedence relationships between tasks in the queen's genotype.

Step 5: Calculate the fitness of brood1 and brood2; choose the better one as the final brood.

Taking the problem (P9) in Fig. 2 as an example, a crossover process is illustrated in Fig. 4.

5.5. Workers

In the original algorithm, workers are not separate members of the population but used as search techniques to improve the

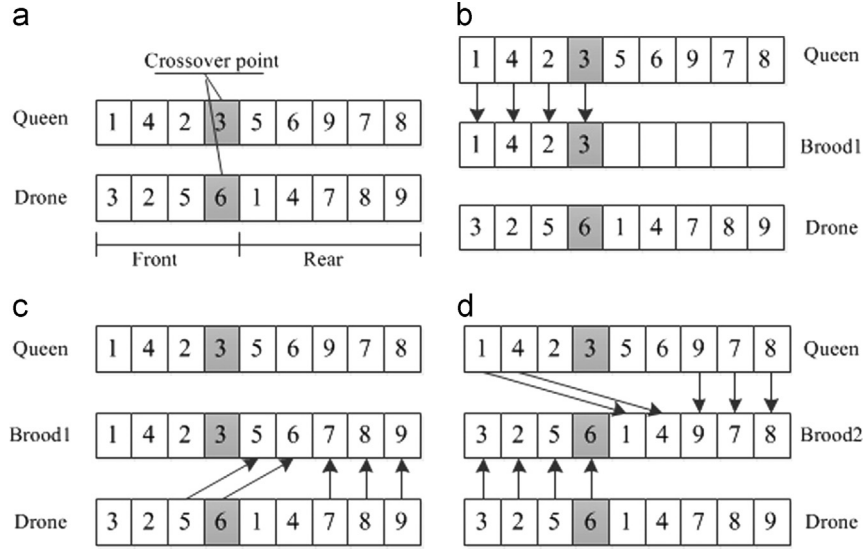


Fig. 4. Illustration of crossover operator.

broods generated by the mating flight of the queen [24]. In this study, SA with three different neighborhood structures is regarded as three kinds of workers and adopted in Step 3. The steps of SA with one neighborhood structure follow the standard framework of SA, and more details can be referred to [35,36]. The steps of improving one brood by the worker are described as follows.

- Step 1: Select one brood generated in the crossover process.
- Step 2: Select a neighborhood structure randomly.
- Step 3: Use SA with the selected structure to improve the brood.

The three neighborhood structures used in SA comply with the precedence relationships so that infeasible solutions cannot be generated. And they are backward-insert, forward-insert and adjacent-swap. Supposing that i, j denote the i th and j th gene in the genotype, and $h(i)$ represents the corresponding task in the i th gene of a given genotype.

The first one is called *backward-insert*:

- Step 1: Randomly generate an integer i , and set $j=i+1$.
- Step 2: Check the precedence relationship between task $h(i)$ and $h(j)$. If $h(i)$ should be performed before $h(j)$ or j is greater than the length of the genotype, then execute Step 3; otherwise, set $j=j+1$, and execute Step 2 again.
- Step 3: Insert task $h(i)$ to the location of $h(j-1)$.

The second one is called *forward-insert*:

- Step 1: Randomly generate an integer i , and set $j=i-1$.
- Step 2: Check the precedence relationship between task $h(i)$ and $h(j)$. If $h(i)$ should be performed before $h(j)$ or j is smaller than zero, then execute Step 3; otherwise, set $j=j-1$, and execute Step 2 again.
- Step 3: Insert task $h(i)$ to the location of $h(j+1)$.

The third one is called *adjacent-swap*:

- Step 1: Randomly generate an integer i , and set $j=i+1$.
- Step 2: Check the precedence relationship between task $h(i)$ and $h(j)$. If $h(i)$ should be performed before $h(j)$, then execute Step 2; otherwise, execute Step 3.
- Step 3: Swap the two corresponding tasks.

Taking the genotype in Fig. 3 as an example, the above three neighborhood structures are illustrated in Fig. 5.

6. Numerical experiment

In order to evaluate the performance of the proposed algorithm, the numerical experiment is performed in the paper. All the tests are conducted on a personal computer with an Intel Core 2 Duo 2.93 GHz CPU and 2 GB memory. And HHBMO is coded in C++ language by the software of Microsoft Visual C++ 6.0. In the experiment, each instance is run five times [20]. The details and results of the experiment are presented in following subsections.

6.1. Test problems and compared algorithms

Seven categories of problems with various cycle times are solved: four small-sized problems, P9, P12, P16, and P24; three large-sized problems, P65, P148 and P205. P9, P12 and P24 are taken from Kim et al. [9], P16, P65 and P205 are taken from Lee et al. [3], and P148 is taken from Bartholdi [2]. The precedence relations and operation directions of the tasks of those problems are not changed, task times of the first six problems are taken from Ozcan and Toklu [20], and task times of P205 are randomly generated for each model between zero and the maximum value of the original task times, which are listed in Table A.2 of Supplementary data. And the overall proportions of the number of units of all models are the same ($q_A=q_B=\dots=q_m$).

The results of the HHBMO algorithm are compared with those of MIP and SA for small-sized problems, and those of SA for large-sized problems [20]. The details of the MIP model can be referred to [20], the model is solved by using the GAMS (General Algebraic Modeling System) mathematical programming package, and the runs which are not finished are interrupted at 7200s. In the simulated annealing (SA) algorithm, the priority values of tasks are applied for encoding, the swap and insert operators are used as neighborhood structures [20]. The initial temperature, the final temperature, the Markov chain length, and the cooling rate of SA are 1000, 0.001, the number of tasks of the problems, and 0.99, respectively [20].

6.2. Parameter setting

The number of parameters of the proposed algorithm which is composed of HBMO and SA is ten. HBMO contains six parameters: number of queens, number of drones, number of mating flights, size

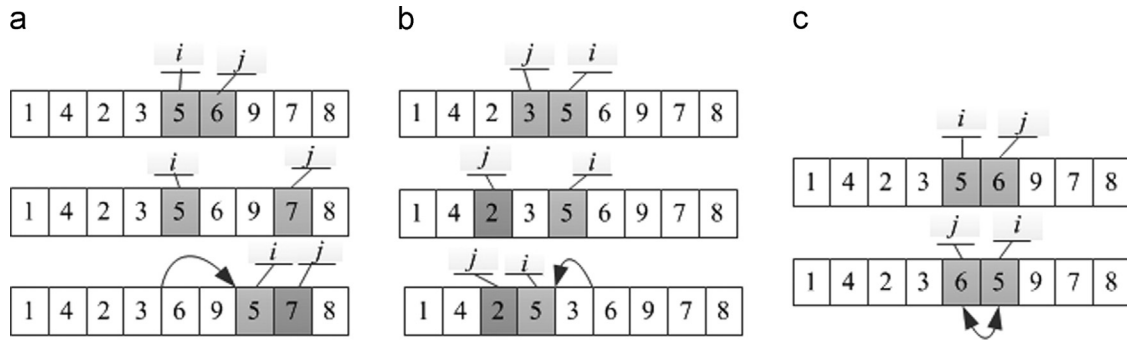


Fig. 5. Illustration of three neighborhood structures.

Table 1
Parameters of the HHBMO algorithm.

Parameters	Values	
	Small-sized	Large-sized
Number of queens	1	1
Number of drones	100	300
Number of mating flights	10	10
Size of queen's spermatheca	15	25
Initial speed and energy	10,000.0	10,000.0
Speed and energy coefficient	0.90	0.90
Initial temperature	1000.0	1000.0
Markov chain length	20	40
Cooling rate	0.90	0.96
Final temperature	0.001	0.001

of queen's spermatheca, initial speed and energy, and speed and energy coefficient. And SA includes four parameters introduced in Section 6.1. Due to so many parameters of the algorithm, we firstly determine six of them according to some applications of HBMO and SA to other problems [20,24,25,32]. The six parameters are number of queens, number of mating flights, initial temperature and speed, speed and energy coefficient, initial temperature, and final temperature. Their values can be referred to Table 1. For the rest of parameters, including number of drones, size of queen's spermatheca, Markov chain length, and cooling rate, their test values are {100, 200, 300, 400}, {10, 15, 20, 25, 30}, {20, 30, 40, 50}, and {0.90, 0.92, 0.94, 0.96, 0.98}, respectively. Every combination of parameters is tested ten times on total five instances which are selected according to the number of tasks. They are P16 with $CT=18$, P24 with $CT=30$, P65 with $CT=381$, P148 with $CT=357$, and P205 with $CT=2454$. Note that the optimality and calculation speed have conflicting manner, and we should trade-off between the two goals. For example, if the size of the queen's spermatheca increases, the optimality of solutions will enhance with the cost of lower calculation speed of the algorithm. Therefore, we can find good parameters by try-and-error. And Table 1 shows the complete parameters of the algorithm for small-sized and large-sized problems.

In fact, the parameters for large-sized instances can also be applied for small-sized ones. However, using them for small-sized instances spends more than eight times as much time as using current ones, but achieves the same number of mated-stations and total number of stations for the same instance according to our tests (see Table A.1 of Supplementary data). So for the sake of saving the running time, we adopt different parameters for small-sized and large-sized instances.

6.3. Results and discussions

Table 2 gives computational results and comparison of HHBMO with MIP and SA for small-sized problems. As the size of the

problem increases, MIP cannot be solved by the software in a reasonable computational time. So Table 3 only gives computational results and comparison of HHBMO with SA for large-sized problems. The last column (i.e., Relative gap of WSI) of Table 2 is calculated by Eq. (9). And the last column (i.e., Relative gaps) of Table 3 is calculated by Eqs. (10) and (9). In the equations, WSI_{SA} and WLE_{SA} denote the mean WSI and WLE obtained by SA, WSI_{HHBMO} and WLE_{HHBMO} denote the mean WSI and WLE obtained by HHBMO, respectively. Table 4 gives the computational results and lower bounds for P205. All the lower bounds in Tables 2–4 are calculated according to the formulas (11) and (12) [20]. Only nonzero values are displayed in the tables.

$$\text{Relative gap of WSI} = (WSI_{SA} - WSI_{HHBMO}) / WSI_{SA} \quad (9)$$

$$\text{Relative gap of WLE} = (WLE_{HHBMO} - WLE_{SA}) / WLE_{SA} \quad (10)$$

$$Max = \max \left\{ \left| \frac{\sum_{h \in A_L m \in M} q_m t_h^m}{CT} \right|, \left| \frac{\sum_{h \in A_R m \in M} q_m t_h^m}{CT} \right| \right\} \quad (11)$$

$$LB = 2Max + \max \left\{ 0, \left| \frac{\sum_{h \in A_L m \in M} q_m t_h^m - (Max \cdot CT) - \sum_{h \in A_R m \in M} q_m t_h^m - (Max \cdot CT) - \sum_{h \in A_R m \in M} q_m t_h^m}{CT} \right| \right\} \quad (12)$$

In Table 2, we compare the performance of HHBMO with that of MIP and SA for small-sized problems. The results of MIP and SA are from Ref. [20]. In terms of the number of mated-stations, total number of stations and the weighted line efficiency (WLE), HHBMO achieves the same results as MIP and SA do. However, HHBMO outperforms MIP and SA in terms of the weighted smoothness index (WSI). It obtains 18 out of 19 solutions to small-sized instances better than SA. The largest and average relative gap of WSI obtained by the two algorithms can reach 65.12% and 32.46%, respectively. The average running time of all small-sized problems is about 4.42 s.

In Table 3, we compare the performance of HHBMO with that of SA for large-sized problems to demonstrate its search ability. The results of SA are from Ref. [20]. For total 12 large-sized instances (5 instances of P65 and 7 instances of P148), HHBMO gets 1 and 5 better solutions than SA in terms of the number of mated-stations and total number of stations, respectively. And this is very important for industries to save the fixed manufacturing cost including tools, fixtures, and operators. The same 5 better solutions have higher WLE, which means better utilization of assembly lines. Moreover, all the values of WSI got by HHBMO are also better than those of SA. Most of the differences are from 50% to 80%, as shown in Fig. 6. The largest relative gap of WSI obtained by the two algorithms can reach 81.78%. And lower WSI means better balanced workloads among stations, which reduces workers'

Table 2

Comparison with other approaches for small-sized problems.

Problem	Models	CT	LB	MIP ^a		nm[ns]		SA ^a		HHBMO								Relative gap of WSI (%)
										nm[ns]		WLE		WSI		CPU (s)		
										Best	Mean	Best	Mean	Best	Mean			
P9	2	4	4	3[4]		3[4]		78.12	1.275	3[4]	3[4]	78.12	78.12	0.791	0.791	2.89	37.96	
		5	3	2[3]		2[3]		83.33	1.080	2[3]	2[3]	83.33	83.33	0.577	0.577	2.47	46.57	
		6	3	2[3]		2[3]		69.44	1.080	2[3]	2[3]	69.44	69.44	0.577	0.577	2.46	46.57	
P12	2	5	5	3[5]		3[5]		84.00	1.183	3[5]	3[5]	84.00	84.00	0.707	0.707	3.57	40.24	
		6	4	2[4]		2[4]		87.50	0.866	2[4]	2[4]	87.50	87.50	0.707	0.707	3.34	18.36	
		7	3	2[4]		2[4]		75.00	0.866	2[4]	2[4]	75.00	75.00	0.707	0.707	3.30	18.36	
		8	3	2[3]		2[3]		87.50	1.291	2[3]	2[3]	87.50	87.50	1.291	1.291	3.10		
		15	5	4[6]		4[6]		75.00	3.708	4[6]	4[6]	75.00	75.00	3.651	3.651	4.66	1.54	
		16	5	4[6]		4[6]		70.31	4.735	4[6]	4[6]	70.31	70.31	3.651	3.651	4.51	22.89	
		18	4	3[5]		3[5]		75.00	5.908	3[5]	3[5]	75.00	75.00	5.797	5.797	4.35	1.88	
		19	4	3[5]		3[5]		71.05	5.908	3[5]	3[5]	71.05	71.05	5.797	5.797	4.30	1.88	
		21	4	2[4]		2[4]		80.35	5.160	2[4]	2[4]	80.35	80.35	3.937	3.937	4.20	23.70	
		22 ^b	4	2[4]		2[4]		76.40	4.861	2[4]	2[4]	76.70	76.70	2.574	2.574	4.22	47.05	
		20	7	4[7]		4[7]		89.28	2.903	4[7]	4[7]	89.28	89.28	1.336	1.833	6.52	36.86	
		24	6	3[6]		3[6]		86.80	2.858	3[6]	3[6]	86.80	86.80	1.414	1.536	6.21	46.26	
		25	5	3[6]		3[6]		83.33	2.799	3[6]	3[6]	83.33	83.33	1.414	1.466	6.17	47.62	
		30	5	3[5]		3[5]		83.33	2.449	3[5]	3[5]	83.33	83.33	0.894	1.148	5.96	53.12	
		35	4	2[4]		2[4]		89.28	2.646	2[4]	2[4]	89.28	89.28	0.500	0.923	5.88	65.12	
		40	4	2[4]		2[4]		78.12	2.291	2[4]	2[4]	78.12	78.12	0.612	0.900	5.82	60.72	

^a MIP and SA are tested on a personal computer with an Intel Pentium 4, 3.0 GHz CPU and 512 MB memory.^b According to our calculation, WLE should be 76.70 when the cycle time is 22. So we only list our result of this instance.**Table 3**

Comparison with SA for large-sized problems.

Problem	Models	CT	LB	SA ^a		HHBMO												Relative gaps (%)		
																		CPU (s)	WLE	WSI
						nm[ns]		WLE		WSI		nm[ns]		WLE		WSI				
						Best	Mean	Best	Mean	Best	Mean	Best	Mean	Best	Mean	Best	Mean			
P65	3	326	8	5[9]	5[9]	84.93	84.93	62.767	74.547	5[9]	5[9]	84.93	84.93	27.372	30.466	288.92		59.13		
		381	7	4[8]	4[8]	81.75	81.75	69.490	74.900	4[8]	4[8]	81.75	81.75	17.148	19.006	313.09		74.62		
		435	6	4[7]	4[7]	81.83	81.83	85.773	91.177	4[7]	4[7]	81.83	81.83	13.596	16.612	349.03		81.78		
		490	6	3[6]	3[6]	84.76	84.76	76.340	87.550	3[6]	3[6]	84.76	84.76	18.751	19.729	289.95		77.47		
		544	5	3[6]	3[6]	76.34	76.34	74.575	81.540	3[6]	3[6]	76.34	76.34	14.742	15.352	272.57		81.17		
P148	4	204	13	9[17]	9[17]	75.81	75.81	60.455	64.840	9[17]	9[17]	75.81	75.81	28.558	32.613	803.51		49.70		
		255	11	7[14]	7[14]	73.64	73.64	77.108	78.877	7[13]	7[13.8]	79.31	74.48	25.686	28.405	752.69	1.14	63.99		
		306	9	6[12]	6[12]	71.60	71.60	86.990	91.337	6[11]	6[11]	78.11	78.11	29.147	32.741	726.07	9.09	64.15		
		357	8	5[10]	5[10]	73.64	73.64	102.764	105.237	5[10]	5[10]	73.64	73.64	17.686	22.031	714.04		79.07		
		408	7	5[10]	5[10]	64.44	64.44	89.030	92.654	4[8]	4[8]	80.55	80.55	15.597	22.301	705.24	25.00	75.93		
		459	6	4[8]	4[8]	71.60	71.60	103.301	109.363	4[7]	4[7.8]	81.83	73.65	17.445	22.383	704.40	2.86	79.53		
		510	6	4[8]	4[8]	64.44	64.44	81.284	84.656	4[7]	4[7]	73.64	73.64	16.305	17.713	697.18	14.28	79.08		

^a SA is tested on a personal computer with an Intel Pentium 4, 3.0 GHz CPU and 512 MB memory.

dissatisfaction with the huge difference between their workloads. The average running times of P65 and P148 are about 302.71 and 729.02 s, respectively.

In Table 4, we compare the performance of HHBMO with the lower bound of each instance, since P205 has not been tested by any other approach. As described in Table 4, the difference between the result of HHBMO and the lower bound is between one and four, and decreases as CT increases, which demonstrates the same change as P148. If the size of P205 and the complex nature of the MTALB problem are taken into consideration, the results achieved by HHBMO can be acceptable. And the average running time of P205 is 1247.89 s (about 20.80 min).

In summary, in terms of total number of stations, five better solutions are from P148, and in terms of WSI, with the increasing size of problems, more solutions having larger relative gap of WSI can be achieved by HHBMO, which means that HHBMO can find

better combinations of tasks in the same or less number of stations compared with SA. It demonstrates that as the size of the MTALB problem increases, MIP and SA are no longer suitable for the problem, and HHBMO is able to produce highly competitive results.

7. Conclusion

In this study, a novel hybrid honey bee mating optimization (HHBMO) algorithm was proposed for solving the mixed-model two-sided assembly line balancing (MTALB) problem with the objective of minimizing the number of mated-stations and total number of stations. A new encoding and decoding scheme was devised to fit the problem, and SA was employed as workers to improve broods in the proposed algorithm. In order to verify the

Table 4
Computational results of P205.

Problem	Models	CT	LB	HHBMO						CPU (s)
				nm[ns]		WLE		WSI		
				Best	Mean	Best	Mean	Best	Mean	
P205	5	1133	11	8[15]	8[15]	70.42	70.42	185.813	195.909	1368.77
		1322	10	7[13]	7[13]	69.63	69.63	205.492	213.805	1313.21
		1510	8	6[11]	6[11.2]	72.05	70.85	207.102	251.055	1277.60
		1699	8	5[10]	5[10]	70.44	70.44	148.804	161.609	1256.12
		1888	7	5[9]	5[9]	70.43	70.43	254.775	265.009	1227.52
		2077	6	4[8]	4[8]	72.02	72.02	188.212	196.961	1238.84
		2266	6	4[8]	4[8]	66.02	66.02	228.932	241.303	1206.96
		2454	6	4[7]	4[7]	69.67	69.67	314.279	321.085	1211.12
		2643	5	3[6]	3.2[6.2]	75.46	73.31	153.747	240.811	1180.32
	2832	5	3[6]	3[6]	70.43	70.43	179.469	187.004	1198.48	

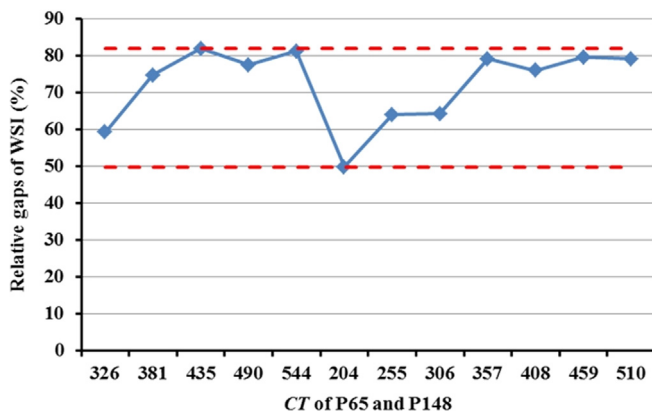


Fig. 6. Relative gaps of WSI for large-sized problems.

effectiveness of the HHBMO algorithm, seven sets of problems were tested, and the results were compared with those of MIP, of SA for small-sized problems, and those of SA for large-sized problems. The experiment results showed that the proposed algorithm achieved good-quality solutions in reasonable computational times, which validated the effectiveness of HHBMO for the problem.

In the future, HHBMO can be applied to solve other combinatorial optimization problems. The encoding and decoding scheme proposed in this paper can also be modified to solve other two-sided assembly line balancing problems.

Acknowledgments

The authors thank three anonymous referees and the area editor whose comments helped a lot to improve this paper. This research work is supported by the State Key Program of National Natural Science of China (Grant no. 51275190), the National Natural Science Foundation of China (Grant no. 51035001), the National Natural Science Foundation of China (Grant No. 51121002), and the Science & Technology Major Project of China (Grant No. 2011ZX04015-011-07).

Appendix A. Supplementary materials

Supplementary data associated with this article can be found in the online version at <http://dx.doi.org/10.1016/j.cor.2014.07.011>.

References

- [1] Chutima P, Chimklai P. Multi-objective two-sided mixed-model assembly line balancing using particle swarm optimisation with negative knowledge. *Comput Ind Eng* 2012;62(1):39–55.
- [2] Bartholdi J. Balancing two-sided assembly lines: a case study. *Int J Prod Res* 1993;31(10):2447–61.
- [3] Lee TO, Kim Y, Kim YK. Two-sided assembly line balancing to maximize work relatedness and slackness. *Comput Ind Eng* 2001;40(3):273–92.
- [4] Simaria AS, Vilarinho PM. 2-ANTBAL: an ant colony optimisation algorithm for balancing two-sided assembly lines. *Comput Ind Eng* 2009;56(2):489–506.
- [5] Scholl A, Becker C. State-of-the-art exact and heuristic solution procedures for simple assembly line balancing. *Eur J Oper Res* 2006;168(3):666–93.
- [6] Abbass HA. MBO: marriage in honey bees optimization – a haplometrosis polygynous swarming approached. In: *Proceedings of the 2001 congress on evolutionary computation*; 2001. p. 207–14.
- [7] Boysen N, Fließdner M, Scholl A. A classification of assembly line balancing problems. *Eur J Oper Res* 2007;183(2):674–93.
- [8] Tasan SO, Tunali S. A review of the current applications of genetic algorithms in assembly line balancing. *J Intell Manuf* 2008;19(1):49–69.
- [9] Kim YK, Kim YH, Kim YJ. Two-sided assembly line balancing: a genetic algorithm approach. *Prod Plan Control* 2000;11(1):44–53.
- [10] Lapierre SD, Ruiz AB. Balancing assembly lines: an industrial case study. *J Oper Res Soc* 2004;55(6):589–97.
- [11] Baykasoglu A, Dereli T. Two-sided assembly line balancing using an ant-colony-based heuristic. *Int J Adv Manuf Technol* 2008;36:582–8.
- [12] Wu EF, Jin Y, Bao JS, Hu XF. A branch-and-bound algorithm for two-sided assembly line balancing. *Int J Adv Manuf Technol* 2008;39:1009–15.
- [13] Hu XF, Wu EF, Bao JS, Jin Y. A branch-and-bound algorithm to minimize the line length of a two-sided assembly line. *Eur J Oper Res* 2010;206(3):703–7.
- [14] Hu XF, Wu EF, Jin Y. A station-oriented enumerative algorithm for two-sided assembly line balancing. *Eur J Oper Res* 2008;186(1):435–40.
- [15] Ozcan U, Toklu B. A tabu search algorithm for two-sided assembly line balancing. *Int J Adv Manuf Technol* 2009;43:822–9.
- [16] Ozcan U, Toklu B. Multiple-criteria decision-making in two-sided assembly line balancing: a goal programming and a fuzzy goal programming models. *Comput Oper Res* 2009;36(6):1955–65.
- [17] Ozcan U, Toklu B. Balancing two-sided assembly lines with sequence-dependent setup times. *Int J Prod Res* 2010;48(18):5363–83.
- [18] Ozcan U, Gokcen H, Toklu B. Balancing parallel two-sided assembly lines. *Int J Prod Res* 2010;48(16):4767–84.
- [19] Ozbakir L, Tapkan P. Bee colony intelligence in zone constrained two-sided assembly line balancing problem. *Expert Syst Appl* 2011;38(9):11947–57.
- [20] Ozcan U, Toklu B. Balancing of mixed-model two-sided assembly lines. *Comput Ind Eng* 2009;57(1):217–27.
- [21] Rabbani M, Moghaddam M, Manavizadeh N. Balancing of mixed-model two-sided assembly lines with multiple U-shaped layout. *Int J Adv Manuf Technol* 2011;59:1191–210.
- [22] Haddad OB, Afshar A, Marino MA. Honey-bees mating optimization (HBMO) algorithm: a new heuristic approach for water resources optimization. *Water Resour Manag* 2006;20(5):661–80.
- [23] Marinakis Y, Marinaki M. A hybrid honey bees mating optimization algorithm for the probabilistic traveling salesman problem. 2009 IEEE Congress on Evolutionary Computation; 2009. p. 1762–69.
- [24] Marinakis M, Marinakis Y, Zopounidis C. Honey bees mating optimization algorithm for financial classification problems. *Appl Soft Comput* 2010;10(3):806–12.
- [25] Marinakis Y, Marinaki M, Dounias G. Honey bees mating optimization algorithm for the Euclidean traveling salesman problem. *Inf Sci* 2011;181(20):4684–98.

- [26] Niknam T, Taheri SI, Aghaei J, Tabatabaei S, Nayeripour M. A modified honey bee mating optimization algorithm for multi-objective placement of renewable energy resources. *Appl Energy* 2011;88(12):4817–30.
- [27] Sabar NR, Ayob M, Kendall G, Qu R. A honey-bee mating optimization algorithm for educational timetabling problems. *Eur J Oper Res* 2012;216(3):533–43.
- [28] Sadjadi SJ, Soltani R. Alternative design redundancy allocation using an efficient heuristic and a honey bee mating algorithm. *Expert Syst Appl* 2012;39(1):990–9.
- [29] Tseng HE. Guided genetic algorithms for solving a larger constraint assembly problem. *Int J Prod Res* 2006;44(3):601–25.
- [30] Leu YY, Matheson LA, Rees LP. Assembly line balancing using genetic algorithms with heuristic-generated initial populations and multiple evaluation criteria. *Decis Sci* 1994;25(4):581–605.
- [31] Boysen N, Fließner M, Scholl A. Sequencing mixed-model assembly lines: survey, classification and model critique. *Eur J Oper Res* 2009;192(2):349–73.
- [32] Ruan Q, Zhang Z, Miao L, Shen H. A hybrid approach for the vehicle routing problem with three-dimensional loading constraints. *Comput Oper Res* 2013;40(6):1579–89.
- [33] Kim YK, Song WS, Kim JH. A mathematical model and a genetic algorithm for two-sided assembly line balancing. *Comput Oper Res* 2009;36(3):853–65.
- [34] Ozcan U. Balancing stochastic two-sided assembly lines: a chance-constrained, piecewise-linear, mixed integer program and a simulated annealing algorithm. *Eur J Oper Res* 2010;205(1):81–97.
- [35] Dowsland K, Thompson J. Simulated annealing. In: Rozenberg G, Back T, Kok J, editors. *Handbook of natural computing*. Germany Berlin Heidelberg: Springer-Verlag; 2012. p. 1623–55.
- [36] Henderson D, Jacobson S, Johnson A. The theory and practice of simulated annealing. In: Glover F, Kochenberger G, editors. *Handbook of metaheuristics*. US, New York: Springer; 2003. p. 287–319.